Overview of Functional Coverage and CRV in Verification

EEL 4712 Spring 2026

HDL vs. SystemVerilog



Limitations of HDLs (VHDL, Verilog)

- Limited support for *abstraction* and *reusability*
  - *Low-level* abstraction (e.g., gate-level and register-transfer): adequate for digital designs and systems of modest complexity
- Limited support for *design verification*
  - VHDL has ASSERT statement

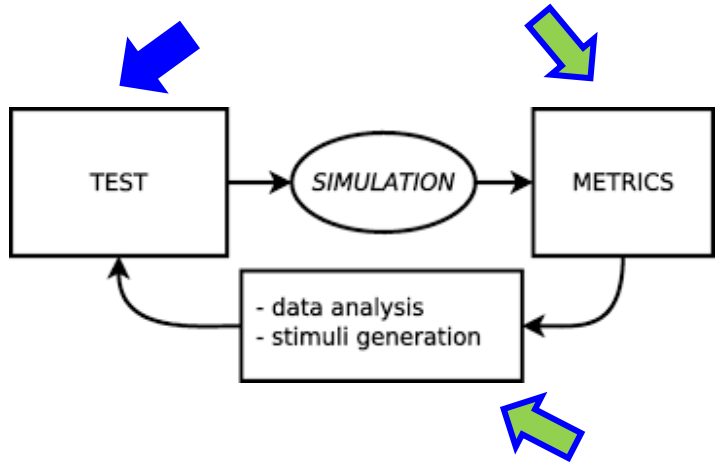


Key desirable features of SystemVerilog

- Enhanced *abstraction*
  - For design and *reuse* (e.g., classes, objects, encapsulation)
- Robust *verification capabilities*
  - Assertation, constrained random stimuli, functional coverage
- *Industry adoption*



Design Verification: Testing and Coverage



Directed Testing, Constrained-Random Testing

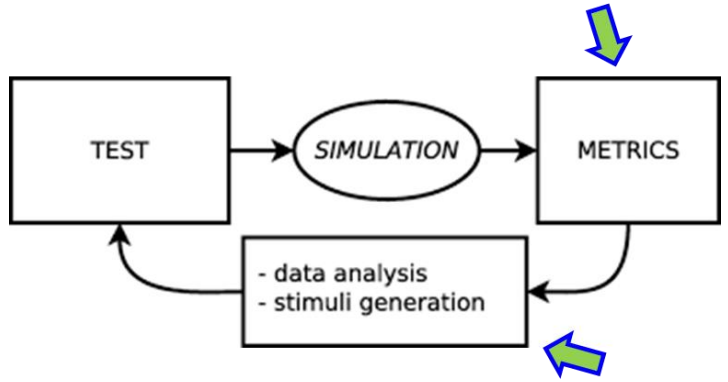
- **Directed tests:** Explicitly define inputs to test specific scenarios to verify certain design features.
- **Random, constrained-random tests**:
  - **Random:** generate a large number of tests with random inputs
  - **Constrained-random:** random, but within user-defined constraints to uncover corner cases and bugs that might be missed by directed or completely random tests.

Coverage:

- a *simulation metric* used to measure the *progress* and *completeness* (quality) of the verification process
  - To provide feedback to improve test

Types of Coverage

- Coverage: a *simulation metric* used to measure the *progress* and *completeness* (quality) of the verification process
- Major types of coverage methods:
  - Functional coverage
  - Code coverage

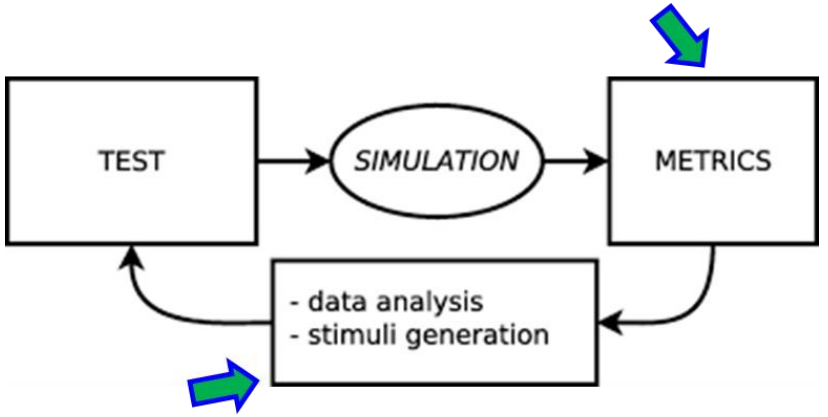


- **Functional** coverage:
  - Measures how much of the functionality defined in the design specification have been exercised during the simulation
  - Specified by the designer in the testbench
- **Code** coverage:
  - Measures how much of the code have been exercised during the simulation
  - Collected automatically by the simulation tool (e.g., Questa)

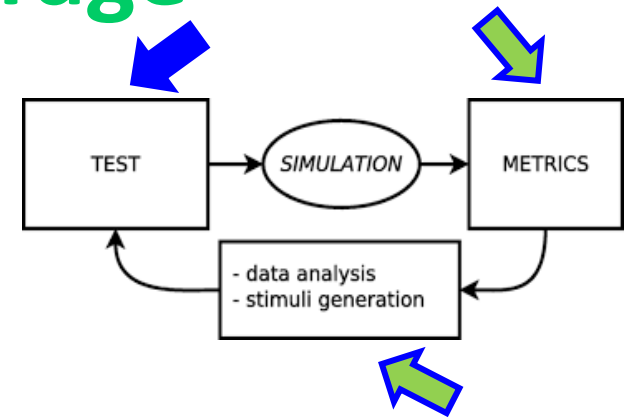
Data-oriented Functional Coverage

Data-oriented function coverage: A ***method*** to collect and record if specified data values, combinations, or patterns have been "covered" during simulation. Summary of method:

- **Define** a *test plan* to define the functionality to be verified, based on the design specification
  - Translate test plan into ***constraints*** and functional ***coverage*** specifications
- **Specify** *in testbenches (TBs)* the specifications
  - **Constraints**
  - **Coverage:** Covergroups, coverpoints, bins
- **During simulation**, the verification environment
  - Sample data of the values of variables specified in TB
  - Collect and compare coverage data in coverage bins
  - Report coverage summary and details to provide **feedback** to improve test



Design Verification: Testing and Coverage



Directed Testing, Constrained-Random Testing

- **Directed tests:** Explicitly define inputs to test specific scenarios to verify certain design features.
- **Random, constrained-random tests**:
  - **Random:** generate a large number of tests with random inputs
  - **Constrained-random:** random, but within user-defined constraints to uncover corner cases and bugs that might be missed by directed or completely random tests.

Coverage:

- a *simulation metric* used to measure the *progress* and *completeness* (quality) of the verification process
  - To provide feedback to improve test

Virtual Dice Example

```
module dice_logic (
    input  logic [4:0] die1,       // Valid inputs 1-20, invalid 0, 21-31  input
    logic [4:0] die2,       // Valid inputs 1-20, invalid 0, 21-31  output logic
    [5:0] sum,          // Sum of die1 and die2
    output logic double_flag,      // 1 if both dice equal
    output logic odd_sum,          // 1 if sum is odd
    output logic even_sum          // 1 if sum is even
);
  always_comb begin  // 4 "major" functions to be tested
     sum       = die1 + die2;     // represents "major" function 1
       double_flag = (die1 == die2);   // represents "major" function 2
       odd_sum     = (sum % 2);   // represents "major" function 3
       even_sum    = ~odd_sum;   // represents "major" function 4
  end
endmodule
```





Dice Example Testbench 1 (Snippet)

```
// ---------------------------------
// Functional Coverage
// ---------------------------------
covergroup cg @(posedge clk);
    coverpoint die1 {
      bins valid_faces[]   = {[1:20]};
      bins invalid_faces[] = {0,[21:31]};
    }
    coverpoint die2 {
      bins valid_faces[]   = {[1:20]};
      bins invalid_faces[] = {0,[21:32]};
    }
  endgroup
  cg coverage = new();
```

```
logic [4:0] die1, die2;
...
 // Randomize and test multiple times
   repeat (50) begin
      assert(stim.randomize()) else $fatal("Random failed!");
      // Apply to DUT
      die1 = stim.die1_val;
      die2 = stim.die2_val;
      @(posedge clk);
      // Functional Coverage sample
      coverage.sample();

       ...

   end   // 50 times
```

Questa Demo

Dice Testbench 1 (again, more detailed look)

Go through dice_tb1.sv code in detail

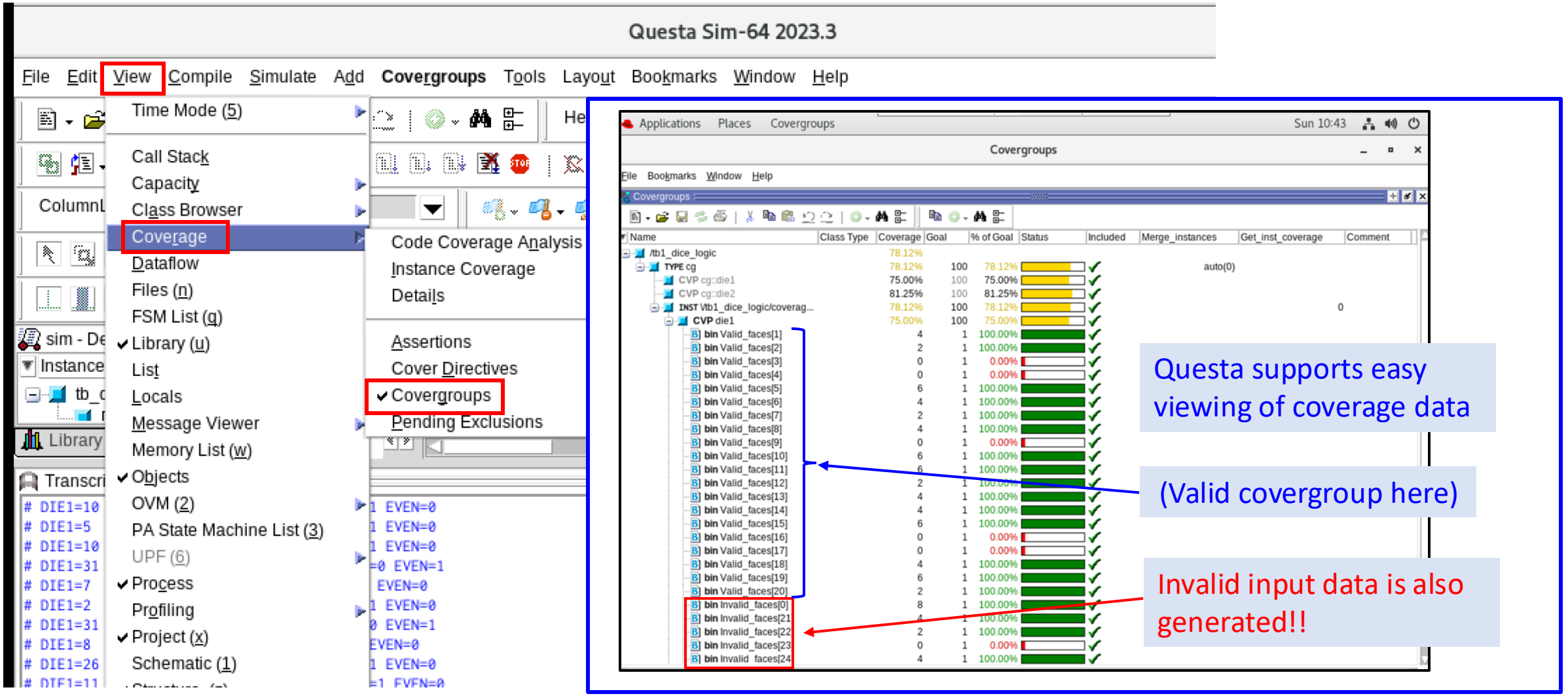
- **Advantage:** Easily generate a specified number of *tests* with random inputs and *sample* and *collect* coverage data.

```
class DiceStimulus;
    rand int die1_val;
    rand int die2_val;
```

```
DiceStimulus stim = new();
assert(stim.randomize()) else ... error msg
die1 = stim.die1_val;
die2 = stim.die2_val;
@(posedge clk);
// Functional Coverage sample
coverage.sample();
```

- **Advantage:** Questa supports easy viewing of coverage data; feedback to improve the testing/validation (See next slide)
- **Disadvantage:** Included invalid input data (See next slide)

Questa Support for Coverage Analysis



Constrained Random Verification (CRV)

- **Directed Testing:** challenging and time-consuming for complex designs
- **Complete randomization** (e.g., Dice TB1): invalid data can be generated

Constrained Random Verification (CRV)

- Technique to automatically generate randomized test cases with specific constraints
- Objectives: To generate *a large number of randomized* test cases to cover a wide range of scenarios
- Challenge: ensure the generated input stimuli meet key design requirements

Coverage

- a *simulation metric* used to measure the *progress* and *completeness* (quality) of the verification process
- To provide feedback to improve test

Dice Example Testbench 2 (Snippet)

```
// ---------------------------------
// Functional Coverage
// ---------------------------------
covergroup cg @(posedge clk);
   coverpoint die1 {
     bins valid_faces[]   = {[1:20]};
     bins invalid_faces[] = {0,[21:31]};
   }
   coverpoint die2 {
     bins valid_faces[]   = {[1:20]};
     bins invalid_faces[] = {0,[21:32]};
   }
 endgroup
 cg coverage = new();
```

```
logic [4:0] die1, die2;
...
 // Randomize and test multiple times
   repeat (50) begin
      assert(stim.randomize()) else $fatal("Random failed!");
      // Apply to DUT
      die1 = stim.die1_val;
      die2 = stim.die2_val;
      @(posedge clk);
      // Functional Coverage sample
      coverage.sample();

       ...

   end   // 50 times
```

Same as dice_tb1.sv

Questa Demo

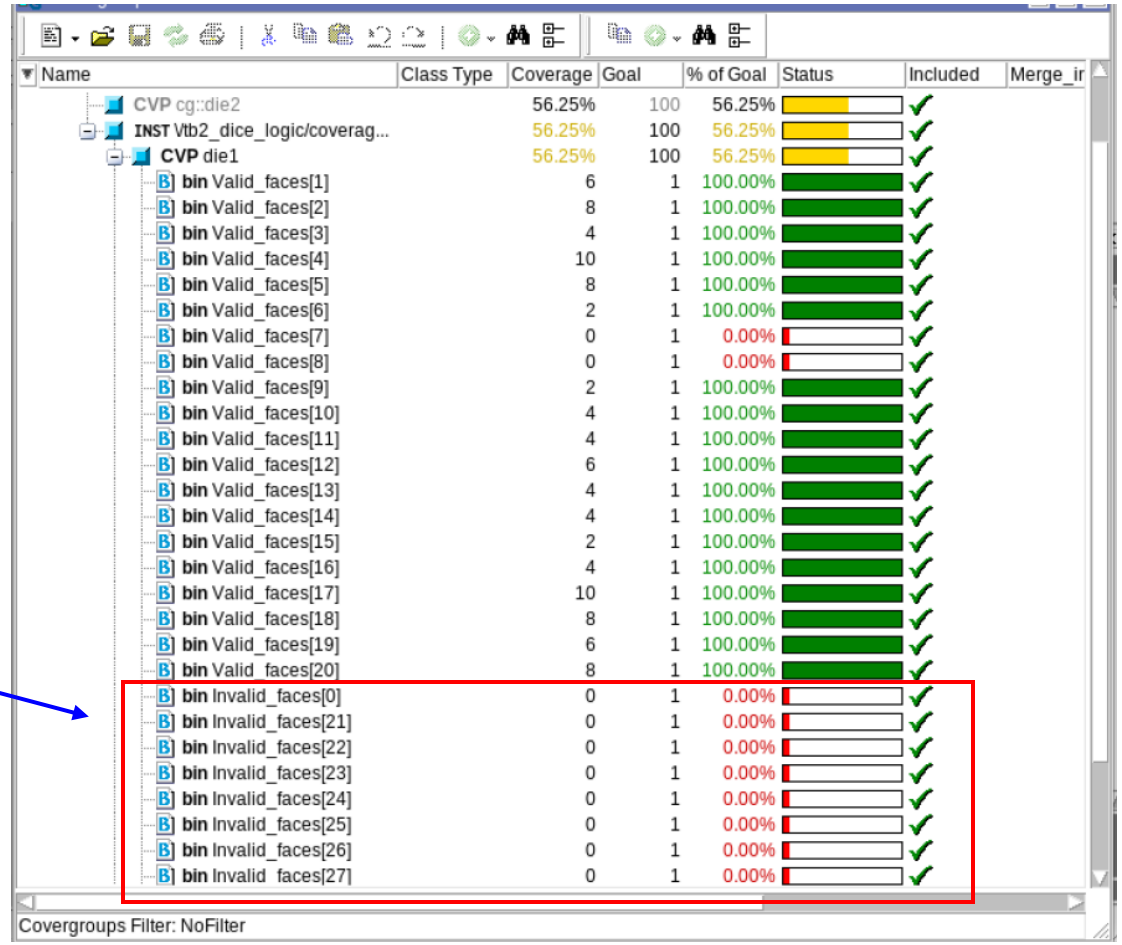
```
// --- Legitimate values constraint
   constraint legit_c { die1_val inside {[1:20]}; die2_val inside {[1:20]}; }
```

constraint statement

// --- Legitimate values constraint

constraint legit_c { die1_val **inside** {[1:20]}; die2_val **inside** {[1:20]}; }

Unlike Testbench 1, there are no input values generated for the variables in the Invalid_faces bins

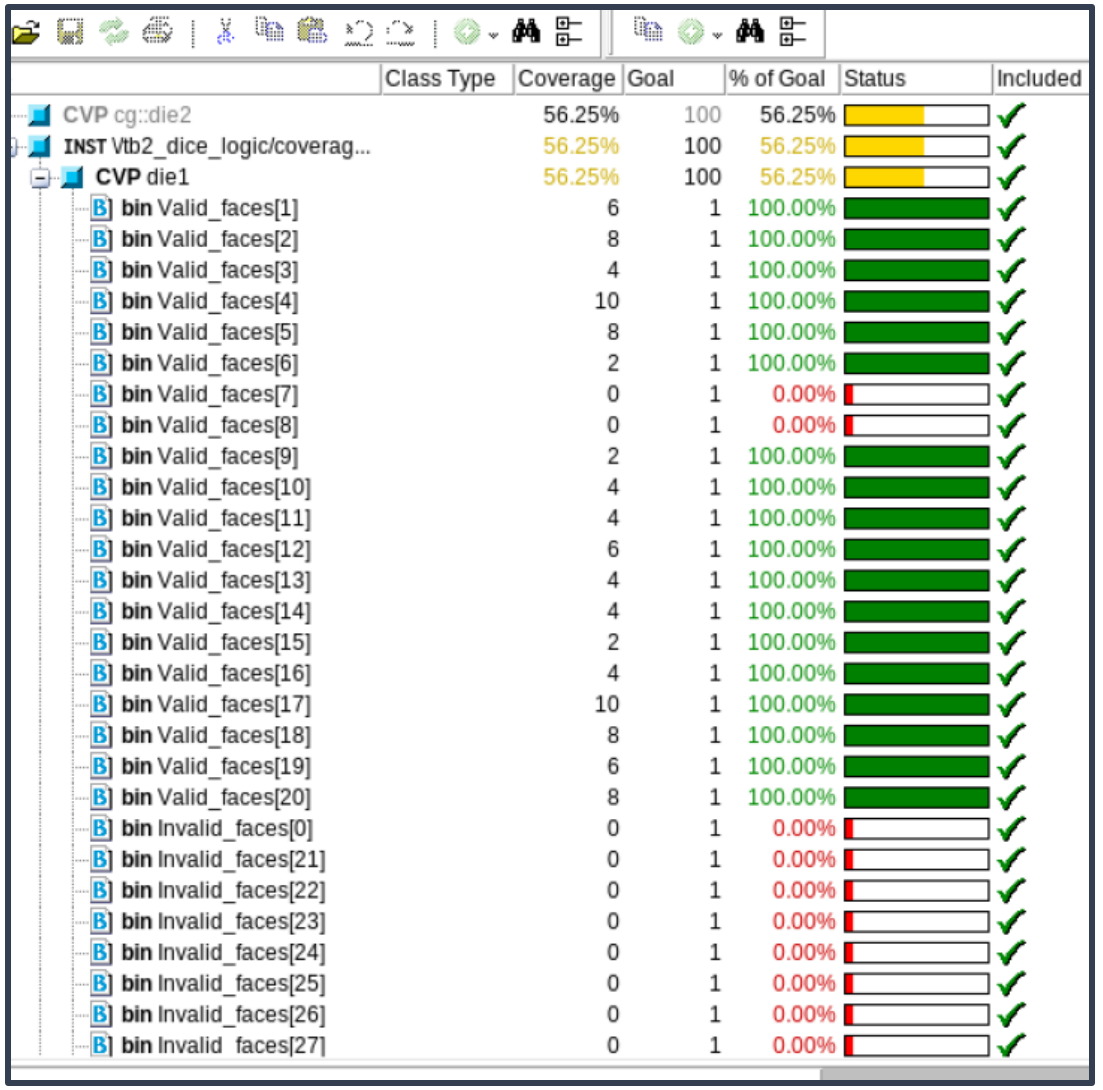


Ignoor_bins
Illegal_bins

// Weighted constraint favor high probability rolls

```
constraint weighted_c {
    die1_val dist {[1:4] := 7, [5:15] := 3, [16:20] := 7};
    die2_val dist {[1:4] := 7, [5:15] := 3, [16:20] := 7};
}
```

- The output for “repeat 50” does not show much.
- Demo dice_tb2.sv again and “repeat 1000”.



	Class Type	Coverage	Goal	% of Goal	Status	Included
CVP cg::die2		56.25%	100	56.25%		
INST Vtb2_dice_logic/coverag...		56.25%	100	56.25%		
CVP die1		56.25%	100	56.25%		
bin Valid_faces[1]		6	1	100.00%		
bin Valid_faces[2]		8	1	100.00%		
bin Valid_faces[3]		4	1	100.00%		
bin Valid_faces[4]		10	1	100.00%		
bin Valid_faces[5]		8	1	100.00%		
bin Valid_faces[6]		2	1	100.00%		
bin Valid_faces[7]		0	1	0.00%		
bin Valid_faces[8]		0	1	0.00%		
bin Valid_faces[9]		2	1	100.00%		
bin Valid_faces[10]		4	1	100.00%		
bin Valid_faces[11]		4	1	100.00%		
bin Valid_faces[12]		6	1	100.00%		
bin Valid_faces[13]		4	1	100.00%		
bin Valid_faces[14]		4	1	100.00%		
bin Valid_faces[15]		2	1	100.00%		
bin Valid_faces[16]		4	1	100.00%		
bin Valid_faces[17]		10	1	100.00%		
bin Valid_faces[18]		8	1	100.00%		
bin Valid_faces[19]		6	1	100.00%		
bin Valid_faces[20]		8	1	100.00%		
bin Invalid_faces[0]		0	1	0.00%		
bin Invalid_faces[21]		0	1	0.00%		
bin Invalid_faces[22]		0	1	0.00%		
bin Invalid_faces[23]		0	1	0.00%		
bin Invalid_faces[24]		0	1	0.00%		
bin Invalid_faces[25]		0	1	0.00%		
bin Invalid_faces[26]		0	1	0.00%		
bin Invalid_faces[27]		0	1	0.00%		

Content Covered In-Class but NOT on Slides

- Bins
  - Ignore_bins (same syntax as bins, bin with goal of 0)
  - illegal_bins (same syntax as bins, bin with a goal of 0, issue error when hit)
- Options that can be applied to coverpoints or entire covergroups
  - option.at_least = 3 (new number of target hits)
  - option.goal = 90 (the target coverage is 90% instead of 100%)